CASO PRÁCTICO – MÓDULO 1

Fundamentos de Java y Programación Orientada a Objetos

⏱ Duración estimada: 16 horas

1. Contexto del módulo

Este módulo constituye el punto de partida del proyecto práctico del plan formativo, en el que se comenzará el desarrollo inicial del sistema NovaBank Digital Services.

En esta primera fase el objetivo es construir una versión inicial del sistema de gestión bancaria, centrándose en la adquisición de los fundamentos técnicos del desarrollo backend en Java. Para mantener el foco en los conceptos básicos de programación, el sistema se implementará como una aplicación de consola interactiva, sin interfaces gráficas ni servicios web.

Las limitaciones iniciales —sin base de datos, sin autenticación, sin red— permiten que el alumno dedique toda su atención a comprender la lógica del dominio bancario y los fundamentos del lenguaje Java, que son la base sobre la que se construirán los módulos posteriores.

1.1 Alcance del sistema en este módulo

En esta primera versión el sistema será utilizado por empleados del banco encargados de realizar operaciones básicas de gestión. Las funcionalidades principales incluirán:

- Registro y búsqueda de clientes.
- Creación y consulta de cuentas bancarias.
- Ejecución de operaciones financieras básicas: depósitos, retiradas y transferencias.
- Consulta de saldos e historial de movimientos.

Para simplificar el desarrollo inicial, la aplicación funcionará completamente en consola. No se implementará autenticación de usuarios y el almacenamiento de datos se realizará en memoria usando estructuras Java estándar.

Nota sobre el almacenamiento en memoria

Almacenar los datos en memoria significa que al cerrar la aplicación todos los datos se pierden. Esto es completamente intencionado en este módulo. El objetivo es que el alumno aprenda a modelar entidades y lógica de negocio sin distracciones arquitectónicas. La persistencia real se incorporará en el Módulo 2.

2. Funcionalidades de la aplicación

La aplicación será interactiva en consola. Al iniciar se mostrará un menú principal y el usuario navegará por los distintos submenús seleccionando opciones numéricas.

2.1 Menú principal

```
=====================================
   NOVABANK - SISTEMA DE OPERACIONES
=====================================
1. Gestión de clientes
2. Gestión de cuentas
3. Operaciones financieras
4. Consultas
5. Salir

Seleccione una opción: _
```

2.2 Gestión de clientes

```
--- GESTIÓN DE CLIENTES ---
1. Crear cliente
2. Buscar cliente
3. Listar clientes
4. Volver
```

2.2.1 Crear cliente

Permite registrar un nuevo cliente en el sistema. El sistema solicitará los datos del cliente por consola y los validará antes de guardarlos.

Datos solicitados:

- Nombre (obligatorio)
- Apellidos (obligatorio)
- DNI / NIF (obligatorio y único)
- Email (obligatorio, único y formato válido)
- Teléfono (obligatorio y único)

Salida esperada en caso de éxito:

```
Cliente creado correctamente.
ID cliente: 1023
```

Salida esperada en caso de error:

```
ERROR: Ya existe un cliente con el DNI 12345678A.
```

Consejo de implementación

Para validar el formato de email, se puede usar una expresión regular sencilla o simplemente comprobar que el String contiene '@' y al menos un punto después de él. No es necesario una validación exhaustiva en este módulo.
Para generar el ID del cliente, se puede usar un contador estático que se incrementa con cada cliente creado. En módulos posteriores, este ID lo generará la base de datos.

2.2.2 Buscar cliente

Permite buscar un cliente ya registrado. El sistema ofrecerá dos criterios de búsqueda:

- Por DNI
- Por ID interno

Salida esperada:

```
Cliente encontrado:
ID:       1001
Nombre:   Juan Pérez
DNI:      12345678A
Email:    juan@email.com
Teléfono: 600123123
```

```
ERROR: No se encontró ningún cliente con DNI 99999999Z.
```

2.2.3 Listar clientes

Muestra todos los clientes registrados en el sistema en formato tabular. Si no hay clientes registrados, se mostrará un mensaje informativo.

```
ID   | Nombre           | DNI        | Email                | Teléfono
-----|------------------|------------|-----------------------------------
1001 | Juan Pérez       | 12345678A  | juan@email.com       | 666666661
1002 | Ana García       | 87654321B  | ana@email.com        | 666666662
```

2.3 Gestión de cuentas

```
--- GESTIÓN DE CUENTAS ---
1. Crear cuenta
2. Listar cuentas de cliente
3. Ver información de cuenta
4. Volver
```

2.3.1 Crear cuenta bancaria

Permite crear una nueva cuenta bancaria asociada a un cliente existente. El alumno deberá solicitar el ID del cliente, verificar que existe, y generar automáticamente el número de cuenta.

Datos solicitados:

- ID del cliente titular de la cuenta.

Datos generados automáticamente por el sistema:

- Número de cuenta en formato IBAN simplificado (ES + 20 dígitos).
- Saldo inicial = 0,00 €.
- Fecha de creación = fecha y hora actuales.

Generación del número de cuenta

Para este módulo, el número de cuenta puede generarse concatenando el prefijo 'ES91210000' con un número secuencial de 12 dígitos con ceros a la izquierda. Por ejemplo: ES91210000000000000001, ES91210000000000000002, etc.

En un sistema real, el IBAN se calcula aplicando un algoritmo matemático sobre el número de cuenta. Para este ejercicio, el formato simplificado es suficiente.

```
Cuenta creada correctamente.
Número de cuenta: ES91210000000000000001
Titular:          Juan Pérez (ID: 1001)
Saldo inicial:    0,00 €
```

2.3.2 Listar cuentas de cliente

```
Introduzca ID del cliente: 1001

Cuentas del cliente Juan Pérez:
Número de cuenta          | Saldo
--------------------------|----------
ES91210000000000000001    | 3.500,00 €
ES91210000000000000002    | 1.200,00 €
```

2.3.3 Ver información de cuenta

```
Introduzca número de cuenta: ES91210000000000000001

Número de cuenta: ES91210000000000000001
Titular:          Juan Pérez
Saldo:            3.500,00 €
Fecha de creación: 2024-03-15 10:23:45
```

2.4 Operaciones financieras

```
--- OPERACIONES FINANCIERAS ---
1. Depositar dinero
2. Retirar dinero
3. Transferencia entre cuentas
4. Volver
```

2.4.1 Depositar dinero

Incrementa el saldo de una cuenta en el importe indicado. El depósito genera automáticamente un registro de movimiento de tipo DEPOSITO.

Datos solicitados:

- Número de cuenta
- Cantidad a depositar

Validaciones:

- La cuenta debe existir en el sistema.
- La cantidad debe ser mayor que cero.

```
Depósito realizado correctamente.
Cuenta:     ES9121000000000000001
Importe:    +1.000,00 €
Nuevo saldo: 3.500,00 €
```

2.4.2 Retirar dinero

Reduce el saldo de una cuenta en el importe indicado. Genera un registro de movimiento de tipo RETIRO.

Validaciones:

- La cuenta debe existir.
- La cantidad debe ser mayor que cero.
- El saldo disponible debe ser suficiente para cubrir el importe solicitado.

```
ERROR: Saldo insuficiente.
Saldo disponible: 200,00 €
Importe solicitado: 500,00 €
```

2.4.3 Transferencia entre cuentas

Mueve un importe de una cuenta origen a una cuenta destino. Es la operación más compleja del sistema porque involucra dos cuentas y debe dejar el sistema en un estado consistente.

Datos solicitados:

- Número de cuenta origen
- Número de cuenta destino
- Cantidad a transferir

Validaciones:

- Ambas cuentas deben existir.
- La cuenta origen y la cuenta destino no pueden ser la misma.
- La cantidad debe ser mayor que cero.
- La cuenta origen debe tener saldo suficiente.

Proceso de ejecución (en este orden exacto):

1. Verificar que ambas cuentas existen y que el saldo es suficiente.
2. Retirar el importe de la cuenta origen.
3. Depositar el importe en la cuenta destino.
4. Registrar un movimiento TRANSFERENCIA_SALIENTE en la cuenta origen.
5. Registrar un movimiento TRANSFERENCIA_ENTRANTE en la cuenta destino.

> **Importancia del orden de operaciones**
> Las cinco acciones de la transferencia deben ejecutarse siempre en el orden indicado. Si se produce cualquier error en el paso 3, 4 o 5, el saldo ya habrá sido modificado en el paso 2. En este módulo, al trabajar en memoria, se puede revertir el cambio manualmente en el bloque catch. En el Módulo 2, este problema se resolverá con transacciones JDBC.

```
Transferencia realizada correctamente.
Cuenta origen:  ES91210000000000000001 → -500,00 €
Cuenta destino: ES91210000000000000002 → +500,00 €
```

2.5 Consultas

```
--- CONSULTAS ---
1. Consultar saldo
2. Historial de movimientos
3. Movimientos por rango de fechas
4. Volver
```

2.5.1 Consultar saldo

```
Introduzca número de cuenta: ES91210000000000000001

Saldo actual: 3.500,00 €
```

2.5.2 Historial de movimientos

Muestra todos los movimientos de una cuenta ordenados del más reciente al más antiguo.

```
Historial de movimientos - ES91210000000000000001:

Fecha                | Tipo                    | Cantidad
---------------------|-------------------------|------------
2024-03-15 14:30:00  | TRANSFERENCIA_SALIENTE  | -500,00 €
2024-03-15 11:00:00  | DEPOSITO                | +1.000,00€
2024-03-15 10:23:45  | DEPOSITO                | +2.500,00€
```

Tipos de movimiento que el sistema debe manejar:

- DEPOSITO: ingreso de dinero en la cuenta.
- RETIRO: extracción de dinero de la cuenta.
- TRANSFERENCIA_SALIENTE: dinero enviado a otra cuenta.

- TRANSFERENCIA_ENTRANTE: dinero recibido de otra cuenta.

2.5.3 Movimientos por rango de fechas

Permite filtrar el historial de movimientos de una cuenta entre dos fechas. El alumno introducirá la fecha de inicio y la fecha de fin, y el sistema mostrará únicamente los movimientos comprendidos en ese rango.

```
Número de cuenta: ES91210000000000000001
Fecha inicio (yyyy-MM-dd): 2024-03-01
Fecha fin    (yyyy-MM-dd): 2024-03-15

Movimientos del 2024-03-01 al 2024-03-15:
...
```

3. Diseño del modelo de datos

Aunque en este módulo la aplicación no utiliza una base de datos, los alumnos deben diseñar el modelo de datos completo del sistema desde el principio. Este modelo se utilizará en módulos posteriores cuando el sistema evolucione hacia una arquitectura con persistencia real.

Diseñar el modelo de datos antes de programar es una práctica fundamental en el desarrollo de software. Obliga al programador a pensar en las entidades del sistema, sus atributos y sus relaciones antes de escribir ninguna línea de código.

3.1 Entidades del sistema

El sistema NovaBank trabaja con tres entidades principales: Cliente, Cuenta y Movimiento.

Tabla CLIENTES

Campo	Tipo SQL	Descripción
id	SERIAL	PK, autogenerada
nombre	VARCHAR(100)	Obligatorio
apellidos	VARCHAR(150)	Obligatorio
dni	VARCHAR(20)	Único y obligatorio
email	VARCHAR(150)	Único y obligatorio. Formato válido.
telefono	VARCHAR(20)	Único y obligatorio
fecha_creacion	TIMESTAMP	

Tabla CUENTAS

Campo	Tipo SQL	Descripción
id	SERIAL	PK, autogenerada
numero_cuenta	VARCHAR(34)	IBAN simplificado, obligatorio y único
cliente_id	INT	FK → clientes.id
saldo	NUMERIC(15,2)	Inicialmente 0. Siempre >= 0
fecha_creacion	TIMESTAMP	

Tabla MOVIMIENTOS

Campo	Tipo SQL	Descripción
id	SERIAL	PK, autogenerada
cuenta_id	INT	FK → cuentas.id
tipo	VARCHAR(50)	DEPOSITO, RETIRO, TRANSFERENCIA_SALIENTE, TRANSFERENCIA_ENTRANTE
cantidad	NUMERIC(15,2)	Siempre > 0
fecha	TIMESTAMP	

3.2 Relaciones entre entidades

- Un Cliente puede tener N Cuentas (relación 1:N).
- Una Cuenta puede tener N Movimientos (relación 1:N).
- Un Movimiento pertenece a exactamente una Cuenta.

Modelo de datos vs. clases Java

El modelo de datos describe cómo se almacenarán los datos en la base de datos (en módulos futuros). Las clases Java del módulo 1 deben reflejar este modelo, pero con las adaptaciones propias de la POO: por ejemplo, la clase Cuenta puede tener un campo 'cliente' de tipo Cliente (referencia al objeto), mientras que en la base de datos se almacenará como 'cliente_id' (solo el identificador).

4. Implementación en memoria

En este módulo los datos se almacenarán en memoria usando estructuras de datos de la API estándar de Java. Cuando la aplicación se cierre, todos los datos se perderán.

4.1 Estructuras de datos recomendadas

Para almacenar los datos en memoria se recomienda usar Map con el identificador como clave, ya que permite búsquedas en O(1):

```
// Almacén de clientes: clave = ID del cliente
Map<Long, Cliente> clientes = new HashMap<>();

// Almacén de cuentas: clave = número de cuenta
Map<String, Cuenta> cuentas = new HashMap<>();

// Almacén de movimientos: clave = ID del movimiento
Map<Long, Movimiento> movimientos = new HashMap<>();

// Contadores para generar IDs autoincrementales
private long contadorClientes    = 1000L;
private long contadorCuentas     = 1L;
private long contadorMovimientos = 1L;
```

4.2 Scripts SQL (preparación para módulos futuros)

Aunque la aplicación no usa base de datos en este módulo, el alumno debe crear los scripts SQL que crearán las tablas en el Módulo 2. Estos scripts serán situados en un archivo src/main/resources/schema.sql Esto es un ejercicio de diseño, no de ejecución.

5. Testing

El proyecto deberá incluir pruebas automatizadas que verifiquen el correcto funcionamiento de las funcionalidades implementadas. Las pruebas se desarrollarán con JUnit 5 y, cuando sea necesario aislar dependencias, con Mockito.

5.1 Qué probar

No es necesario cubrir el 100% del código, pero sí las operaciones críticas del sistema. Las pruebas deben cubrir tanto los casos de éxito como los casos de error, según las validaciones propias de cada campo.

5.2 Ejemplo de test con JUnit 5

```
@Test
void depositar_conImportePositivo_debeActualizarSaldo() {
    // Arrange
    Cuenta cuenta = new Cuenta("ES91210000000000000001", 1001L);
    cuentaRepository.guardar(cuenta);

    // Act
    operacionService.depositar("ES91210000000000000001", new BigDecimal("500"));

    // Assert
    Cuenta actualizada =
cuentaRepository.buscarPorNumero("ES91210000000000000001");
    assertEquals(new BigDecimal("500.00"), actualizada.getSaldo());
}
```

```
@Test
void depositar_conImporteCero_debeLanzarExcepcion() {
    assertThrows(IllegalArgumentException.class, () ->
        operacionService.depositar("ES9121000000000000001", BigDecimal.ZERO)
    );
}
```

5.3 Ejecución de los tests

```
# Ejecutar todos los tests
mvn test
```

6. Uso de Maven

El proyecto deberá utilizar Apache Maven como herramienta de gestión de dependencias y automatización del ciclo de construcción.

6.1 Dependencias del pom.xml

```
   <dependencies>

        <!-- JUnit 5 -->
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>5.10.0</version>
            <scope>test</scope>
        </dependency>
        <!-- Mockito para mocks opcionales en tests-->
        <dependency>
            <groupId>org.mockito</groupId>
            <artifactId>mockito-core</artifactId>
            <version>5.4.0</version>
            <scope>test</scope>
        </dependency>

    </dependencies>

    <build>
        <plugins>

            <!-- Compilador -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.11.0</version>
            </plugin>
```

```
            <!-- Ejecutar aplicación -->
            <plugin>
                <groupId>org.codehaus.mojo</groupId>
                <artifactId>exec-maven-plugin</artifactId>
                <version>3.1.0</version>
                <configuration>
                    <mainClass>com.novabank.Main</mainClass>
                </configuration>
            </plugin>

            <!-- Ejecutar tests -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.2.5</version>
            </plugin>

            <!-- Crear JAR ejecutable -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-jar-plugin</artifactId>
                <version>3.3.0</version>
                <configuration>
                    <archive>
                        <manifest>
                            <mainClass>com.ejemplo.Main</mainClass>
                        </manifest>
                    </archive>
                </configuration>
            </plugin>

        </plugins>
    </build>
```

6.2 Comandos Maven

Comando	Qué hace
mvn compile	Compila el código fuente de src/main/java
mvn test	Ejecuta todas las pruebas unitarias
mvn package	Genera el archivo .jar en el directorio target/
mvn exec:java	Ejecuta la aplicación de consola
mvn clean	Elimina el directorio target/ con los compilados anteriores

Orden de trabajo recomendado

Durante el desarrollo, alterna entre 'mvn compile' (verificar que compila) y 'mvn test' (verificar que los tests pasan). Usa 'mvn exec:java' para probar la consola interactiva. Reserva 'mvn clean package' para las entregas. 8. Control de versiones con Git El código fuente deberá alojarse en un

7. Control de versiones con Git

El código fuente deberá alojarse en un repositorio público en GitHub. El alumno deberá mantener un flujo de trabajo ordenado durante todo el desarrollo.

7.1 Estructura de ramas

Rama	Propósito
main	Versión estable del proyecto
develop	Integración de funcionalidades en desarrollo
feature/clientes	Desarrollo de gestión de clientes
feature/cuentas	Desarrollo de gestión de cuentas
feature/operaciones	Desarrollo de operaciones financieras
feature/consultas	Desarrollo de consultas e historial

7.2 Convención de commits

Cada commit debe representar una unidad lógica de trabajo y tener un mensaje descriptivo. Se recomienda seguir el estándar Conventional Commits:

```
feat: implementación del modelo de dominio (Cliente, Cuenta, Movimiento)
feat: creación y búsqueda de clientes en memoria
feat: operación de depósito con validación de importe
feat: transferencia entre cuentas con registro de movimientos
fix: corrección de validación de saldo insuficiente en retirada
test: pruebas unitarias para ClienteService
test: pruebas unitarias para OperacionService
docs: actualización del README con instrucciones de ejecución
```

8. README del repositorio

El repositorio deberá incluir un archivo README.md que permita a cualquier desarrollador comprender el proyecto y ejecutarlo en su entorno local. A continuación se describe el contenido mínimo que debe tener.

Secciones mínimas del README

- Descripción del proyecto: qué es NovaBank y qué hace la aplicación en este módulo.
- Tecnologías utilizadas: Java 17, Maven, JUnit 5, Mockito, Git.
- Requisitos del sistema: versión de Java y Maven necesarias para ejecutar el proyecto.
- Cómo compilar: mvn clean compile
- Cómo ejecutar: mvn exec:java
- Cómo ejecutar los tests: mvn test

- Estructura del proyecto: explicación de los paquetes y su responsabilidad.
- Enlace al repositorio de GitHub.

9. Documento PDF entregable

Al finalizar el módulo, el alumno entregará un documento en PDF que documente el trabajo realizado. El documento deberá incluir las siguientes secciones:

- Introducción: descripción del sistema NovaBank y del contexto del módulo.
- Descripción funcional: explicación de cada funcionalidad implementada, con ejemplos de uso por consola.
- Modelo de datos: descripción de las entidades, sus campos y sus relaciones. Incluir un diagrama entidad-relación.
- Decisiones de implementación: cómo se ha implementado el almacenamiento en memoria y por qué se han elegido las estructuras utilizadas.
- Scripts SQL: el archivo schema.sql con explicación de cada tabla.
- Testing: descripción de los tests implementados, con capturas de la ejecución exitosa de mvn test.
- Git: capturas del historial de commits y de la estructura de ramas del repositorio.
- Enlace al repositorio público en GitHub.